WORKSHOP M2 — 2026

RESILIENCE-ALPHA SHADOW-SETTLE

Système de paiement fictif à confidentialité cryptographique, détection d'anomalies par machine learning et remédiation autonome.

> Ce que ce dossier prouve. Une attaque lancée depuis un poste distinct est détectée par un modèle entraîné sur le trafic du poste, la remédiation s'exécute sans intervention humaine, une transaction de preuve passe ensuite, et l'attaquant est dérouté vers un honeypot — en 6,7 à 7,7 secondes (limite 90 s).

19 services Docker	221 tests automatisés	12 failles fermées	7 s de reprise mesurée

Workshop2026-M2-G1 — dossier d'ingénierie et de gouvernance
Version du 22 octobre 2026 — mesures exécutées sur le poste de démonstration

1. Résumé exécutif

RESILIENCE-ALPHA est un système d'information de paiement fictif, conteneurisé, dont la propriété centrale est de se rétablir seul après une attaque. Le scénario n'est pas simulé : un poste distinct attaque le point d'entrée public, le détecteur franchit un seuil appris, un playbook Ansible contient l'incident, une transaction réelle prouve la reprise, puis l'attaquant est dérouté vers un honeypot.

Preuve	Mesure du 22/10	Où la vérifier
Chaîne P0 (attaque → reprise)	7,2 / 7,4 / 6,7 s — trois exécutions consécutives	`python tools/e2e_demo.py`
Détection	seuil appris 6,715 ; FPR 0 % ; rappel 100 %	`services/ml-detector/threshold.json`
Boutique	commande `paid` en 72 ms, key image stockée	`python tools/e2e_shop.py`
Attaques	4 familles + 6 familles boutique, 10/10 conformes	`python tools/attack.py all` / `shop`
Résilience aux pannes	6 scénarios de chaos, 6 PASS	`python tools/chaos.py`
Tests	221 (202 pytest, 5 script, 9 Rust, 10 HTTP)	`./tools/run_tests.sh`
IA locale	RAG BM25, 18 documents, 709 passages, `ollama:llama3.1:8b`	`curl localhost:8091/healthz`
Observabilité	12 cibles Prometheus `up`, 24 règles + 4 règles de logs	`localhost:9090`, `localhost:3001`

Trois décisions qui expliquent le reste

- Le seuil est appris, pas choisi. Un seuil codé en dur serait éliminatoire au regard du sujet. Le modèle est entraîné sur une capture du trafic réel du poste (1 287 observations), et le seuil est fixé sur des fenêtres hors échantillon.
- La détection est hors du chemin critique. Si le détecteur tombe, le paiement continue : le ML observe, il ne filtre pas.
- L'état a une seule source. Le moteur publie (`StateBus`), le dashboard et le terminal consomment. Deux logiques parallèles finiraient par diverger le jour de la démonstration.

> Limite assumée, écrite dès la page 1. Ce système n'est pas « invulnérable » : 12 failles ont été trouvées et fermées, 4 bugs critiques corrigés, et 10 limites restent ouvertes (Bulletproofs, vivacité anti-photo, TLS, AppRole Vault, PostgreSQL mono-nœud…). Une limite écrite ne coûte pas de point ; une limite découverte en Q&A en coûte.

2. Contexte, périmètre et méthode

2.1 Le problème

Une plateforme de paiement subit des incidents à répétition. Chaque fois, les équipes la relancent à la main : l'intervention est lente, non reproductible et impossible à prouver. Le sujet impose donc trois choses qui changent l'architecture : une attaque réelle depuis un poste distinct, une détection par machine learning et non par seuils fixes, et une remédiation automatique suivie d'une preuve — une nouvelle transaction qui réussit.

2.2 Périmètre livré

Pilier	Composants	Ce qui le rend réel
DEV	`payment-api`, `monero-crypto-core`, `front-dashboard`	noyau Rust bLSAG appelé à chaque transaction
INFRA	Docker Compose, Ansible, Prometheus, Alertmanager, Loki, Grafana	playbook réellement exécuté dans l'image
IA/DATA	`ml-detector` (scikit-learn + PyTorch)	seuil appris, anti-rebond, AUC mesurée
COMMERCE	`shop-api`, PostgreSQL, vitrine, back d'administration	articles en base, paiement réel, historique
IA/RAG	`rag-assistant` (BM25, Ollama optionnel)	réponse sourcée, mode extractif explicite
CYBER	`waf`, `shadow-api`, `auth-service`, Vault, `backoffice`	déception active, honeypot, 2FA visage
TRANSFORMATION	dossier EBIOS/PCA-PRA/NIS2/DORA	risques reliés aux composants réels

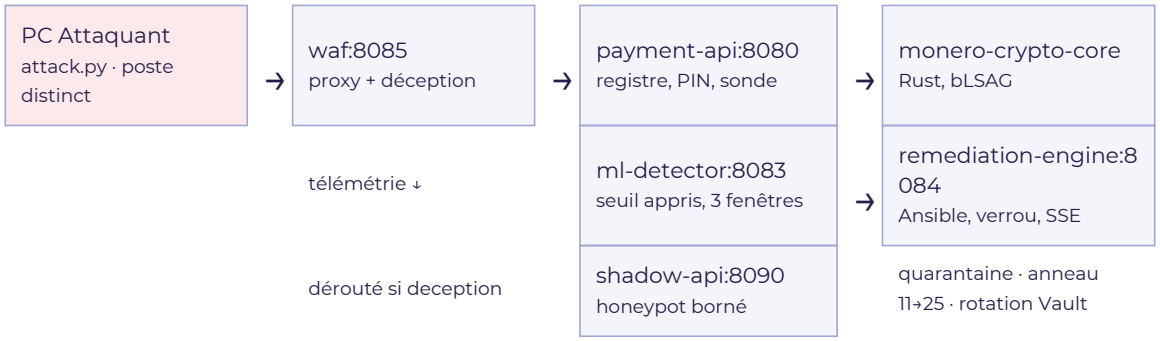
2.3 Méthode

Chaque brique a été livrée avec un test qui échoue si la propriété disparaît. Les mesures ne sont pas des estimations : elles viennent d'exécutions du script de recette, sur la pile Docker du poste de démonstration. Quand une mesure n'a pas été refaite sur ce poste, le document le dit explicitement — c'est le cas des anciens chiffres conservés pour mémoire.

> Le risque dominant n'est pas l'attaque, c'est la panne silencieuse. Sur les défauts corrigés pendant le projet, aucun n'était une attaque réussie : tous venaient d'une erreur d'exploitation ou d'une dépendance matérielle (image sans modèle, webhook cassé, verrou jamais réarmé, fenêtre non purgée, alerte SIEM ciblant le mauvais service).

3. Architecture

Le flux nominal traverse six composants. Le WAF est le seul point d'entrée « client » : l'attaquant y passe, le client légitime aussi — mais seule la transaction qui porte le signal d'attaque est déroutée.



3.1 Ce qui relie réellement les piliers

Un pilier n'est pas « présent » parce qu'un conteneur porte son nom : il l'est quand un flux le traverse. C'est ce que vérifie `tools/e2e_demo.py` : le ML publie un incident au moteur, le moteur exécute Ansible, Ansible appelle le WAF et `payment-api`, Vault tourne la clé JWT, et le dashboard consomme le bus SSE.

3.2 Décisions structurantes

- Le terminal physique ne peut pas casser la démo. Le pont est strictement en lecture, aucun abonné SSE n'est obligatoire, et aucun chemin ne va du terminal vers le moteur. Un test vérifie l'absence de POST.
- Le WAF ne déclenche rien. Sa règle de bord est volontairement grossière : seul le détecteur déclenche, seul le playbook arme la déception. Une déception qui casse le paiement du client serait pire que pas de déception.
- Le registre est persistant. Outputs, key images et portefeuille sont dans PostgreSQL et rechargés au démarrage — sinon la moitié des rejeux échouait après un redémarrage.

4. Détection : apprendre, pas seuiller

Le sujet exclut explicitement « de simples seuils fixes ». Le pipeline est donc entièrement appris, et la mesure a imposé quatre corrections de conception.

4.1 Le pipeline

- `features.py` calcule six features, avec une seule définition partagée entre l'entraînement et l'inférence.
- `train.py` apprend un `StandardScaler`, un `IsolationForest` et un autoencodeur PyTorch, puis fixe le seuil sur des fenêtres hors échantillon.
- `detector.py` compare le score au seuil appris et exige trois fenêtres consécutives anormales avant de déclencher.

4.2 Ce que la mesure a corrigé

Défaut	Symptôme mesuré	Correction
Entropie calculée sur les âges poolés	3,0 → 5,4 bits à distribution identique	calcul par événement puis moyenne
`inter_arrival` en écart-type	une pause de 6 s produisait un score de 6,2	médiane des intervalles
Sonde de liveness dans la télémétrie	cadence fixe de 4 s, distribution biaisée	champ `synthetic` exclu
Entropie comme feature principale	AUC 0,56 : sature à 10 leurres	âge médian des leurres (AUC 1,00)

4.3 Séparation mesurée sur 1 285 fenêtres

Feature	Trafic normal	Attaque (leurres empoisonnés)	AUC
`decoy_median_age_days`	3,610 ± 0,180 j	26,920 ± 2,337 j	1,000
`ring_age_skewness`	0,885 ± 0,212	1,362 ± 0,289	0,920
les 4 autres	—	—	0,500 (bruit pour cette attaque)

Justification physique. Un wallet Monero tire ses leurres selon une gamma biaisée vers les outputs récents : 3,6 j mesurés. Un attaquant qui empoisonne les leurres les tire uniformément dans le registre : 26,9 j. Aucun chevauchement. C'est un résultat honnête à assumer : une seule feature discrimine l'attaque principale, et elle est physiquement justifiée. Les cinq autres couvrent les autres familles (rejeu, rafale, anneau malformé).

> Seuil appris : 6,715 — FPR 0 % (0 % sur trois fenêtres consécutives), rappel 100 % sur 1 285 fenêtres normales et 1 285 fenêtres d'attaque. Aucune constante d'exploitation ne subsiste dans le code : `grep -r "threshold.*=.*[0-9]" services/ml-detector/*.py` ne renvoie rien.

5. Cryptographie : un modèle Monero simplifié mais réel

Le noyau `monero-crypto-core` est écrit en Rust sur Ristretto255. Il ne simule pas une signature : il en produit une, et la vérifie. Deux écarts avec Monero de production sont publiés dans `/healthz` et assumés ici.

Primitive	Implémentation
Adresse one-time	`P = H_s(rA)G + B`, `R = rG` (ECDH)
Clé dérivée par le porteur	`x = H_s(aR) + b`
Key image (anti-double-dépense)	`I = x·H_p(P)`, hash-vers-le-groupe (Elligator) : pas de log discret connu, donc pas de forge
Signature d'anneau liée	bLSAG sur n clés, message lié aux engagements
Engagements	`C = aG + rH`, homomorphes, aveugles sommant à zéro
Confidentialité du montant	masque `Keccak("amount" || shared)`
Audit view-only	`GET /audit/last` : lit le montant sans la Spend Key

5.1 Les deux écarts, et pourquoi ils sont acceptables

Écart	Conséquence	Pourquoi c'est assumé
CLSAG au lieu de bLSAG	signature ~2× plus grosse	même modèle de sécurité, aucune conséquence fonctionnelle
Pas de Bulletproofs	un montant négatif modulo l passerait la vérification d'équilibre	exige un attaquant authentifié qui forge ses propres engagements ; publié dans `/healthz`

Ces deux points ne sont pas cachés : ils apparaissent dans `GET /healthz` (`not_implemented`), dans `docs/SECURITE.md` §3 et dans `TODO.md` §1.1 avec un critère d'acceptation. Bulletproofs est la première tâche après le workshop.

5.2 Ce que la démonstration prouve réellement

- un rejeu de la même key image est refusé en 409, y compris sous concurrence (key image réservée avant le premier `await`) ;
- un anneau de 65 leurres est refusé en 422 avant tout calcul ;
- un auditeur lit le montant par la View Key sans pouvoir dépenser — c'est l'argument DORA art. 19 ;
- le montant d'une commande vient de la base, jamais du client.

6. Remédiation autonome et déception

Le playbook est un vrai `ansible-playbook`, exécuté dans l'image du moteur. Il ne redémarre aucun service : un redémarrage destructif n'est pas une remédiation — la version d'origine aurait effacé le registre d'outputs, c'est-à-dire la preuve que la démo doit montrer.

#	Composant	Action réelle
1	`ml-detector`	3 fenêtres anormales → `POST /incident`
2	`remediation-engine`	répond 202, publie `UNDER_ATTACK`
3	(attente 3 s)	le rouge est visible à l'écran et sur l'objet
4	`remediation-engine`	publie `REMEDIATING`, lance Ansible
5	Ansible	déception : `POST waf:8085/admin/redirect`
6	Ansible	quarantaine : 25 outputs les plus anciens
7	Ansible	durcissement : anneau 11 → 25
8	Ansible	rotation de la clé JWT dans Vault
9	Ansible	preuve : `GET /tx/auto-probe/status`
10	`remediation-engine`	publie `RECOVERED_TX` avec la durée

6.1 Pourquoi l'asynchrone

Un playbook réel prend 4 à 8 secondes. La première version appelait ce webhook de façon synchrone avec un timeout de 5 s : le détecteur tombait en `ReadTimeout` alors que la remédiation fonctionnait. `/incident` répond donc immédiatement et le playbook tourne en tâche de fond ; l'état est publié au fil de l'exécution via SSE.

6.2 La déception active

Le honeypot existait mais personne ne lui redirigeait le trafic : l'étape 1 du playbook visait un service inexistant, échouait sous `ignore_errors`, et le playbook « réussissait » sans rien faire. `services/waf` corrige cela : il apprend l'âge des outputs en proxyfiant `/outputs`, et déroute vers `shadow-api` les transactions dont l'âge médian des leurres dépasse 8 jours — uniquement quand la déception est armée par le playbook.

> Le test qui compte : avec la déception armée, une transaction dont les leurres sont récents passe normalement. Une déception qui casse le paiement du client serait pire que pas de déception.

7. Preuves mesurées

7.1 Trois exécutions consécutives de la chaîne P0

Exécution	Détection	Reprise	TX de preuve	Déception	Ansible	Chaîne
1	3,6 s	7,2 s	OK	active	rc=0	complète
2	3,4 s	7,4 s	OK	active	rc=0	complète
3	3,1 s	6,7 s	OK	active	rc=0	complète

Limite fixée : 90 s. Marge : 82 à 83 s. Aucune intervention humaine entre l'attaque et la reprise.

7.2 Attaques rejouables en une commande

Famille	Ce qu'elle exploite	Résultat attendu
`poison`	tirage uniforme des leurres	détection ML, déception, déroutage
`replay`	même output, donc même key image	409, key image consommée une fois
`burst`	cadence machine (50 ms)	détection par `inter_arrival_median`
`malformed`	anneau court, clés invalides	400 propre, aucun panic
`shop` (6 familles)	quantités, panier, SKU, historique, survente, prix	6/6 refusées, 0 commande frauduleuse

7.3 Tests, charge et chaos

- 226 tests : 202 pytest, 5 scénarios de script, 9 Rust, 10 HTTP. Chaque correction de faille a son test de régression nommé.
- Charge : paiement 630 req/s sans rupture ; RAG 336 req/s ; la boutique est le point de rupture (~130 ms par achat, > 1 s à 5 clients) à cause de la signature d'anneau — 0 % d'erreurs.
- Chaos : 6 scénarios, chacun avec un invariant métier, pas seulement un code HTTP. 6/6 PASS sur la pile.

> Le défaut que la mesure a trouvé : `/ask` rendait 0,5 req/s avec 2 318 ms de p95, parce qu'il attendait le timeout d'Ollama à chaque appel et que `/healthz` interrogeait Ollama. Disjoncteur + sonde sans appel réseau : ×670 de débit, p95 divisé par 580. Une relecture de code ne trouve pas ça.

8. Le SI de paiement : vitrine, back, historique

Les règles du 20/10 déplacent le centre de gravité : le SI doit être un site de paiement avec des articles en base, un front, un back, des commandes et un historique — pas seulement un tableau de bord.

Exigence	Livré	Preuve
N1 — vitrine connectée à une base	`shop-api` 8087, PostgreSQL, 6 articles	`GET /products` ; 35 tests
N2 — front qui présente les articles	`Storefront.jsx`, images SVG servies	3 tests d'images
N3 — back d'administration	`AdminShop.jsx`, `/admin/products`, `/admin/orders`	session admin exigée, dernier admin protégé
N4 — paiement réel + historique	`/checkout` → `/tx/pay` bLSAG → `orders`/`payments`/`payment_events`	commande `paid` en 72 ms, key image stockée

8.1 Le parcours d'achat, et ce qui le rend réel

- Achat invité. La vitrine est publique : le client n'a pas de compte. Le ticket porte un nom et une adresse e-mail, et l'historique est protégé par un capability token de 192 bits (`customer_ref`). Sans ce jeton : 404, jamais 403 — pas d'oracle d'existence.
- Le prix vient de la base. Le total est recalculé côté serveur ; un prix envoyé par le client est ignoré.
- Le stock est réservé en transaction (`WHERE stock >= ?`), puis restitué si le paiement échoue. Deux acheteurs sur le dernier article : une commande, un 409, jamais un stock négatif.
- Une commande n'est `paid` qu'après une preuve : output du montant exact, signature bLSAG, key image stockée dans `payments`.
- Le double-clic ne débite pas deux fois : le panier est réclamé atomiquement (`UPDATE … WHERE checked_out_at IS NULL`).

8.2 Ce que l'administration montre

Le back-office boutique affiche les articles (prix, stock, retrait) et les commandes avec le client, le montant, la key image et les événements de paiement. La suppression d'un article est une désactivation : un article commandé doit rester lisible dans l'historique.

> Ce qu'on ne dit pas : « c'est une démo ». Le stock est réellement décrémenté, la commande est réellement payée, et un paiement refusé produit une commande `failed` avec son motif — visible par le client et par l'administrateur.

9. IA : RAG local et ingestion SIEM

9.1 RAG — répondre en citant, pas en inventant

`services/rag-assistant` (8091) construit un index BM25 à la construction de l'image, sur `docs/`, `README.md` et `TODO.md` : 18 documents, 709 passages. `POST /ask` renvoie une réponse avec ses sources (fichier, section, score) et le moteur utilisé.

- Ollama (`llama3.1:8b`) tourne sur le PC hôte, le même poste que la pile : `OLLAMA_URL=http://host.docker.internal:11434`.
- Si Ollama est muet, le service répond en extractif — les phrases du dossier, citées — et le dit dans `/healthz`. Il n'invente pas.
- Un disjoncteur évite d'attendre le timeout à chaque question : c'est ce qui a fait passer le débit de 0,5 à 336 req/s.
- Le quadrant « Assistant RAG » du dashboard rend le dispositif visible à l'écran, pas seulement en `curl`.

9.2 SIEM — une alerte doit agir, pas s'afficher

Prometheus évalue 24 règles, Loki en ajoute 4 sur les journaux. Alertmanager pousse les alertes à `remediation-engine` (`POST /alert`). La liste des alertes actionnables est fermée : une alerte de confort (bandeau LED, compte bloqué) est journalisée sans ouvrir d'incident.

- Corrélation multi-alertes : une panne réelle produit 3 à 5 alertes en cascade ; celles reçues dans les 10 s sont rattachées au même incident (`detail.alerts`), pas cinq incidents distincts.
- Le moteur réutilise exactement le chemin de l'incident ML : même verrou, même playbook, même preuve de reprise.
- Le jeton de service est exigé : sans lui, 503 fail-closed.

> Le défaut F12, trouvé par l'audit : la règle Loki `PaiementRefuse` ciblait `payment-api`, or la ligne est écrite par `shop-api`. L'alerte ne pouvait jamais se déclencher — donc aucune remédiation. Sélecteur corrigé, et `tools/check_compose.py` vérifie désormais la chaîne complète.

10. Sécurité : ce qui est fermé, et ce qui reste ouvert

Le document n'affirme pas que le SI est invulnérable — aucun SI ne l'est. Il liste ce qui a été trouvé, corrigé, et la commande qui le rejoue. 12 failles et 4 bugs critiques ont été fermés, chacun avec un test de régression nommé.

#	Faille	Correction
F1	`/auth/login` acceptait n'importe quel secret	table SHA-256, comparaison à temps constant, 503 fail-closed
F2	`/admin/*` de `payment-api` et du WAF ouverts	`SERVICE_TOKEN` obligatoire
F3	`/incident` et `/state/report` ouverts	jeton de service
F4	PIN `1234` codé en dur	PIN généré, 5 refus → 60 s
F5	anneau non borné (100 000 leurres)	`max_length=64` → 422 avant le handler
F6	`/ingest` du détecteur ouvert	jeton obligatoire (évasion impossible)
F7	entrées non strictes (boutique)	`strict=True`, motif de SKU, 422
F8	double dépense concurrente	key image réservée avant le premier `await`
F9	double débit au double-clic	réclamation atomique du panier
F10	oracle d'existence 404/500	format validé, comparaison en octets, 404 uniforme
F11	PIN non-ASCII → 500 sans compter l'échec	refus compté comme les autres
F12	alerte SIEM morte (mauvais sélecteur)	sélecteur corrigé, contrôle statique

10.1 Les 4 bugs critiques

#	Bug	Effet réel
C1	SQL booléen contre PostgreSQL (`active = 1`)	« Application startup failed », conteneur en boucle
C2	double dépense concurrente (= F8)	propriété centrale contournable par deux `curl`
C4	double débit au double-clic (= F9)	deux commandes et deux paiements
S1	oracle 404/500 (= F10)	500 si la commande existe, 404 sinon

10.2 Limites assumées

Limite	Pourquoi c'est acceptable
Pas de Bulletproofs	exige un attaquant authentifié qui forge ses engagements ; publié
Pas de vivacité anti-photo	le visage est un facteur d'identité, le mot de passe reste exigé
Pas de TLS	démonstration sur `localhost` ; la caméra l'exige
AppRole Vault non provisionné	jeton de service, pas de privilège root
PostgreSQL mono-nœud, sans PITR	volume conservé ; RPO non borné, écrit tel quel
RAG `/ask` sans authentification	les documents sont déjà versionnés dans le dépôt
`/history` du moteur public en lecture	trace de playbook, pas de clé ; le dashboard en a besoin

11. Gouvernance : EBIOS, continuité, NIS2, DORA

11.1 EBIOS RM — les scénarios retenus

Scénario	Source de risque	Mesure
SS1 — empoisonnement des leurres	attaquant externe	ML (AUC 1,00) + déception
SS2 — rejeu de transaction	attaquant externe	key image, 409
SS3 — saturation	attaquant externe	anneau borné, 422
SS4 — panne silencieuse	exploitation	alertes, `/healthz`, chaos
SS5 — perte de l'objet physique	matériel	pont en lecture seule, jamais nécessaire
SS6 — falsification de prix / survente	client malveillant	prix serveur, transaction de stock
SS7 — fuite de l'historique	client malveillant	capability token, 404 uniforme

11.2 Continuité (PCA/PRA)

- RTO agrégé de la chaîne de paiement : 15 s, contre 6,7 à 7,7 s mesurées en reprise automatique. La marge couvre le cas où la remédiation échoue et où l'exploitant redémarre à la main.
- `db-postgres` est le point de rupture : il porte les commandes, les paiements, le stock et le registre d'outputs. Sans `pg_dump` scripté, le RPO n'est pas borné — c'est écrit, pas supposé.
- Un volume Docker n'est pas une sauvegarde : `pg-data` survit à un `docker compose down`, pas à une corruption ou à un `DROP TABLE`.

11.3 NIS2 et DORA

Exigence	Mesure	Preuve
NIS2 21.2.a — analyse de risques	EBIOS RM relié aux composants	§11.1
NIS2 21.2.b — gestion des incidents	détection + remédiation autonome	`e2e_demo.py`
NIS2 21.2.c — continuité	PCA/PRA, RTO par service	§11.2
NIS2 21.2.e — sécurité du développement	226 tests, audit de code	`run_tests.sh`
NIS2 21.2.i — contrôle d'accès	3 rôles, 2FA visage + mot de passe	`backoffice`
NIS2 21.2.j — MFA	back-office couvert ; terminal = PIN seul, assumé	`/healthz`
DORA art. 10 — détection	ML non supervisé, seuil appris	§4
DORA art. 11 — réponse	playbook + déception + preuve	§6
DORA art. 19 — traçabilité	audit view-only, journal d'audit, Loki	`/audit/last`

11.4 Données personnelles (RGPD)

Le système n'est pas « sans données personnelles » : le back-office stocke un descripteur facial (donnée biométrique, art. 4.14), des identifiants, des adresses IP et un journal d'audit. La minimisation est appliquée — l'image JPEG n'est pas conservée, seul le descripteur — mais il manque un registre de traitement, un DPO et une AIPD. C'est écrit dans le dossier de gouvernance plutôt que passé sous silence.

Annexe A — Inventaire des points d'entrée

Port	Service	Rôle	Exposition
8085	waf	point d'entrée client + déception	public (démo)
8080	payment-api	registre, PIN, sonde, preuve	localhost
8081	monero-crypto-core	bLSAG, key image, audit	localhost
8082	auth-service	JWT marchand, clé dans Vault	localhost
8083	ml-detector	score, seuil, incident	localhost
8084	remediation-engine	états SSE, playbook, alertes	localhost
8086	backoffice	visage + mot de passe, habilitations, LED	localhost + hotspot
8087	shop-api	catalogue, panier, commandes, admin	public (démo)
8090	shadow-api	honeypot	réseau Docker
8091	rag-assistant	RAG BM25 + Ollama	localhost
3001	grafana	dashboard, Explore Loki	localhost
9090 / 9093	prometheus / alertmanager	métriques et alertes	localhost

Annexe B — Reproduire les mesures

Toutes les commandes se lancent depuis la racine du dépôt.

Ce que l'on prouve	Commande
La pile est prête	`python tools/preflight.py`
Chaîne P0 complète	`python tools/e2e_demo.py`
Achat de bout en bout	`python tools/e2e_shop.py`
4 familles d'attaques	`python tools/attack.py all`
6 attaques boutique	`python tools/attack.py shop`
6 scénarios de chaos	`python tools/chaos.py`
Suite complète	`./tools/run_tests.sh`
Séparation des features	`cd services/ml-detector && python analyze_separation.py`
Assistant RAG	`python tools/warmup_rag.py`
Charge	`python tools/load_test.py --target payment --clients 20`

Annexe C — Volumétrie

Élément	Valeur
Services Docker	19
Tests	221 (202 pytest, 5 script, 9 Rust, 10 HTTP)
Failles fermées / bugs critiques	12 / 4
Cibles Prometheus	12
Règles d'alerte	24 (Prometheus) + 4 (Loki)

Élément	Valeur
Panneaux Grafana	13, dont 3 PostgreSQL
Index RAG	18 documents, 709 passages
Articles en catalogue	6, en PostgreSQL
Bundle dashboard	209,74 kB (64,76 kB gzip)